

Plano Arquitetural v2.0: AIDD-OPS → AIDD-FACTORY

Meta-Orquestrador Agêntico de Stacks Open Source, Integrações e White-Label

Documento de Viabilidade Técnica e Expansão Arquitetural

Metadados do Projeto:

- Documento:** v2_arquitetura-aidd-ops-factory.md
- Feature:** AIDD-Factory (sucessora natural do AIDD-Ops MVP)
- Estratégia de Dados:** Cenário A: Single-Instance, Multi-Database com Provisionamento Dinâmico
- Governança:** Ecossistema AIDD (Multi-Harness & Quality Gates)
- Versão:** 2.0 (Setembro de 2026)
- Documento anterior:** 06-09-2026_feature-arquitetura-aidd-ops.md (v1)
- Status da v1:** 50% implementado (Fases 1-3 + deploy parcial)

1. Resumo Executivo: Por Que a v2 Existe

A v1 deste documento descreveu um pipeline de 10 fases para transformar dores de negócio em plataformas completas auto-hospedadas. A implementação real (AIDD-Ops MVP) entregou 4 das 10 fases com excelência — mas as 6 fases restantes revelaram uma **lacuna arquitetural fundamental** que não era óbvia na v1:

A lacuna não é de infraestrutura. É de geração de código de integração e frontend.

O AIDD-Ops resolve magnificamente o ciclo: "Como coloco isso em produção?". Mas o pipeline completo exige também resolver: "O que exatamente coloco em produção?" — e essa é uma responsabilidade diferente, que pertence a uma camada distinta.

Decisão Central da v2

Pergunta	Resposta
Devemos expandir o aidd-ops?	NÃO — quebraria seu contrato de determinismo zero-LLM
Devemos criar uma nova ferramenta?	SIM – aidd-factory, a 7ª ferramenta do ecossistema
Por quê separar?	Separation of Concerns: infraestrutura (ops) ≠ geração de código (factory) ≠ geração genérica (generator)

Mapa de Responsabilidades (3 Ferramentas Complementares)

aidd-ops (EXISTENTE)	aidd-factory (NOVA)	aidd-generator (EXISTENTE)
Intake	Intake	Idea -> Código
Curadoria	Gateway Gen	(8 fases)
Sizing	Frontend Gen	

aidd-ops (EXISTENTE)	aidd-factory (NOVA)	aidd-generator (EXISTENTE)
Bootstrap	Docker Gen	Para projetos
DNS/Cloud	Docs Gen	Python avulsos
Deploy	Webhook Gen	
Pre-flight	Orchestrator	
Monitor		
INFRAESTRUTURA	APLICACAO + INTEGRACAO	CODIGO GENERICO
(zero LLM)	(LLM + templates)	(LLM + AST)

2. Auditoria Técnica da v1: O Que Realmente Existe

2.1 AIDD-Ops MVP — Inventário Completo

Componente	Linhas	Status	Stubs
src/core/result.py	163	☐ Produção	Zero
src/core/compose_preflight.py	57	☐ Produção	Zero
src/core/coolify.py	395	☐ Produção*	Zero*
src/core/ssh_runner.py	321	☐ Produção	Zero
src/core/preflight.py	290	☐ Produção	Zero
src/core/cofre_credenciais.py	439	☐ Produção	Zero
src/core/uptime_kuma.py	257	☐ Produção	Zero
scripts/pipeline_ops.py	1.092	☐ Produção	Zero
scripts/phases/01_intake.py	137	☐ Produção	Zero
scripts/phases/02_curadoria.py	98	☐ Produção	Zero
scripts/phases/03_sizing.py	130	☐ Produção	Zero
templates/infra/* (8 compose)	~600	☐ Produção	Zero
templates/infra/nichos/* (5 JSON)	~380	☐ Produção	Zero
data/catalogo_nichos.json	107	☐ Produção	Zero
data/requisitos_recursos.json	116	☐ Produção	Zero
mcps/cloudflare-mcp/server.py	133	☐ Produção	Zero
mcps/docker-mcp/server.py	136	☐ Produção	Zero
apps/intake/app.py	163	☐ Produção	Zero

Componente	Linhas	Status	Stubs
apps/intake/intake_core.py	68	❑ Produção	Zero
schemas/* (3 JSON Schemas)	~156	❑ Produção	Zero
gates/G_OPS_MVP.py	339	❑ Produção	Zero
gates/G_OPS_SSH.py	154	❑ Produção	Zero
ansible/playbooks/hardening.yml	148	❑ Produção	Zero
charts/aidd-ops/*	~100	❑ Produção	Zero
TOTAL	~8.500	Zero stubs	Zero

|* CoolifyManager.orquestrar_stack() em modo real retorna REAL_NAO_IMPLEMENTADO — falha honesta, não stub.

2.2 AIDD-Generator — Inventário Resumido

Fase	Linhas	LLM?	Status
Phase 1: Pesquisador	780	Não	❑ GitHub/HF API real
Phase 2: Analisador	599	Sim	❑ LLM + gates
Phase 3: Designer	615	Sim	❑ 5 subagentes paralelos
Phase 4: Decisor	362	Não	❑ Heurística determinística
Phase 5: Criador	1.020	Não	❑ Scaffold full project
Phase 6: Documentador	544	Não	❑ Pandoc HTML/MD/PDF
Phase 7: Auto-crítica	1.008	Não	❑ Score + roadmap
Phase 8: Implementador	1.093	Sim	❑ AST decomposition + self-healing
TOTAL	~6.021		

2.3 Gap Crítico Identificado

aidd-ops: [Intake] [Curadoria] [Sizing] [Bootstrap] [DNS] [Deploy] [Pre-flight] [Monitor]

aidd-factory: [??] [??] [??] [??] [??] [??] [??] [??]

↑ GERAÇÃO DE CÓDIGO DE APLICAÇÃO E INTEGRAÇÃO ↑

O pipeline original de 10 fases mapeia para:

Fase v1	Ferramenta Atual	Status
1. Intake & Diagnóstico	aidd-ops (Fase 1)	❑
2. Curadoria da Stack	aidd-ops (Fase 2)	❑
3. Sizing	aidd-ops (Fase 3)	❑
4. Bootstrap VPS	aidd-ops (SSHRunner)	❑
5. DNS Cloudflare	aidd-ops (Cloudflare MCP)	⚠ Parcial
6. Artefatos & Segredos	aidd-ops (CofreCredenciais)	⚠❑ Falta init.sh dinâmico

Fase v1	Ferramenta Atual	Status
7. Hub de Integração & Frontend	NÃO EXISTE	□
8. Deploy Docker	aidd-ops (Coolify)	△ Parcial
9. Pre-flight	aidd-ops (PreflightRunner)	□
10. Backups & Monitoramento	aidd-ops (UptimeKuma)	△ Falta backup R2

3. Por Que NÃO Expandir o AIDD-Ops

3.1 O Contrato de Determinismo

O AIDD-Ops foi projetado com um contrato inviolável:

Zero LLM. Zero síntese criativa. 100% determinístico.

Cada fase é validada por gates AST que verificam a ausência de chamadas LLM, alucinações ou decisões criativas. Expandir o aidd-ops para gerar código de integração (gateway FastAPI, frontend Next.js) **quebraria esse contrato** porque:

- Geração de código exige LLM** — não existe forma determinística de gerar business logic a partir de requisitos de negócio em linguagem natural
- Templates estáticos não resolvem** — cada nicho tem fluxos de integração diferentes (webhook de agendamento ≠ webhook de pedido)
- O maintainability blast radius** seria enorme — misturar infraestrutura determinística com geração de código LLM-dependente no mesmo módulo torna os gates inúteis

3.2 O Princípio de Responsabilidade Única

Ferramenta	Responsabilidade	Contrato
aidd-ops	"Como coloco em produção?"	Determinístico, zero LLM
aidd-factory	"O que□□□como integro?"	LLM + templates + gates
aidd-generator	"Gero código Python genérico"	LLM + AST + self-healing

Misturar essas responsabilidades criaria um "Frankenstein" difícil de testar, auditar e manter.

4. AIDD-Factory: A 7ª Ferramenta

4.1 Definição

AIDD-Factory é o gerador de código de aplicação e integração para stacks multi-serviço. Ele recebe o PLANO-INFRAESTRUTURA.json (gerado pelo aidd-ops) e produz: gateway de integração, frontend whitelabel, scripts de inicialização, documentação Swagger, e configurações de webhook — tudo pronto para o aidd-ops fazer deploy.

4.2 Arquitetura Interna

aidd-factory/

├─ AGENTS.md # Diretrizes canônicas

├─ scripts/

| └─ pipeline_factory.py # Orquestrador principal (CLI Click)

| └─ contrato_factory.py # Validação de input/output

| └─ phases/

| └─ __init__.py # Registry de fases

| └─ 01_analisador.py # Lê PLANO-INFRAESTRUTURA, extrai requisitos

| └─ 02_gateway.py # Gera código FastAPI do gateway

| └─ 03_frontend.py # Gera scaffold Next.js whitelabel

| └─ 04_docker.py # Gera docker-compose.yml unificado

| └─ 05_init_db.py # Gera init-multiple-databases.sh

| └─ 06_env.py # Gera .env criptografado por serviço

| └─ 07_webhooks.py # Gera endpoints e contratos webhook

| └─ 08_docs.py # Gera Swagger/OpenAPI

| └─ 09_integracao.py # Valida integração cross-service

├─ src/core/

| └─ result.py # Reusa do aidd-ops (componentes compartilhados)

| └─ gateway_generator.py # Engine de geração do gateway

| └─ frontend_generator.py # Engine de geração do frontend

| └─ docker_composer.py # Montador de compose unificado

| └─ swagger_generator.py # Gerador de documentação

├─ templates/

| └─ gateway/

| | └─ main.py.jinja2 # Template FastAPI gateway

| | └─ routes.py.jinja2 # Template de rotas por serviço

| | └─ models.py.jinja2 # Modelos Pydantic

| └─ frontend/

| | └─ next.config.js.j2 # Config Next.js

| | └─ layout.tsx.j2 # Layout whitelabel

| | └─ page.tsx.j2 # Dashboard page

| | └─ tenant.config.json.j2 # Config de marca

| └─ docker/

| | └─ compose.unified.j2 # Compose unificado multi-serviço

```
| | └─ Dockerfile.app.j2 # Dockerfile do gateway
| └─ docs/
| └─ openapi.json.j2 # Swagger spec
| └─ README.md.j2 # Documentação do deploy
└─ data/
  └─ integracoes.json # Mapeamento de integrações por nicho
  └─ schemas/ # JSON Schemas de validação
    └─ schemas/
      └─ schema_factory_input.json
      └─ schema_factory_output.json
    └─ gates/
      └─ G_FACTORY_COMPOSE.py # Valida compose gerado
      └─ G_FACTORY_INTEGRATION.py # Valida integrações
    └─ tests/
      └─ test_pipeline_factory.py
      └─ test_gateway_generator.py
    └─ ...
```

4.3 As 9 Fases da AIDD-Factory

Fase 1: Análise do Plano (Determinística)

Input: PLANO-INFRAESTRUTURA.json

Output: factory_analysis.json

LLM: Não

O que faz: Extrai nicho, ferramentas, bancos lógicos, VPS sizing. Valida contra JSON Schema. Mapeia ferramentas para templates de integração disponíveis.

Fase 2: Geração do Gateway (LLM + Templates)

Input: factory_analysis.json

Output: src/gateway/ (código FastAPI completo)

LLM: Sim (1 chamada, ~3k tokens)

O que faz: Gera o Micro-Serviço Gateway com:

Endpoints REST por serviço (CRUM proxy)

Webhook receiver padronizado

Rate limiting e auth middleware

Healthcheck endpoints (/healthz por serviço)

Modelos Pydantic tipados

Fase 3: Geração do Frontend Whitelabel (LLM + Templates)

Input: factory_analysis.json + nicho config

Output: frontend/ (scaffold Next.js + Tailwind + shadcn/ui)

LLM: Sim (1 chamada, ~4k tokens)

O que faz: Gera:

Layout unificado com sidebar de navegação

Páginas por módulo (CRM, Atendimento, Agendamento)

tenant.config.json para personalização de marca

BFF routes que consomem o gateway

Componentes shadcn/ui pré-configurados

Fase 4: Geração do Docker Compose Unificado (Determinística)

Input: factory_analysis.json + templates de cada serviço

Output: docker-compose.yml unificado

LLM: Não

O que faz: Merge dos compose templates selected by nicho, com:

Rede unificada aidd_internal

Healthchecks encadeados via depends_on: condition: service_healthy

Resource limits do sizing

Port mapping sem colisões (NIH #16)

Fase 5: Geração do init-multiple-databases.sh (Determinística)

Input: bancos_logicos do plano

Output: init-multiple-databases.sh

LLM: Não

O que faz: Script bash que cria bancos isolados + usuários com privilégios restritos via psql.

Fase 6: Geração do .env Criptografado (Determinística)

Input: factory_analysis.json + schemas de .env de cada serviço

Output: .env por serviço (plain) + .env.enc (encrypted via sops)

LLM: Não

O que faz: Gera senhas de alta entropia (OpenSSL rand), monta .env isolado por serviço, cifra via cofre de credenciais.

Fase 7: Geração de Webhooks (LLM + Templates)

Input: factory_analysis.json + mapeamento de integrações

Output: Contratos de webhook + handler code

LLM: Sim (1 chamada, ~2k tokens)

O que faz: Para cada par de serviços que se comunicam:

Define payload schema (JSON)

Gera webhook receiver no gateway

Gera webhook sender configuration

Cria contratos de validação

Fase 8: Geração da Documentação (Determinística)

Input: gateway code + webhook contracts
Output: openapi.json + README.md
LLM: Não
O que faz: Extrai rotas do gateway FastAPI, gera Swagger/OpenAPI spec, monta README com instruções de deploy.

Fase 9: Validação Cross-Service (Determinística)

Input: Todos os artefatos gerados
Output: Validação pass/fail
LLM: Não
O que faz:
docker compose config (syntaxe)
Validação de portas sem colisão
Verificação de .env completeness
Healthcheck chain validation
Swagger spec validation

5. O Pipeline Completo v2: 15 Fases Cross-Tool

A v1 descrevia 10 fases em uma única ferramenta. A v2 distribui 15 fases em 3 ferramentas com contratos claros:

Fases 1-3: AIDD-Ops (Planejamento Determinístico)

#	Fase	LLM	Output
1	Intake & Diagnóstico	Não	texto → nicho_slug
2	Curadoria da Stack	Não	nicho → ferramentas[]
3	Sizing de Recursos	Não	ferramentas → vps_spec

Fases 4-12: AIDD-Factory (Geração de Aplicação)

#	Fase	LLM	Output
4	Análise do Plano	Não	plano → factory_analysis.json
5	Geração do Gateway	Sim	→ src/gateway/ (FastAPI)
6	Geração do Frontend	Sim	→ frontend/ (Next.js)
7	Compose Unificado	Não	→ docker-compose.yml
8	Init DB Script	Não	→ init-multiple-databases.sh
9	.env Criptografado	Não	→ .env + .env.enc
10	Webhooks & Integrações	Sim	→ webhook handlers
11	Documentação Swagger	Não	→ openapi.json + README

#	Fase	LLM	Output
12	Validação Cross-Service	Não	→ pass/fail

Fases 13-15: AIDD-Ops (Deploy & Operação)

#	Fase	LLM	Output
13	Bootstrap VPS	Não	VPS hardened
14	DNS + Deploy	Não	Serviços no ar
15	Pre-flight + Monitor	Não	Healthz + Uptime Kuma

6. Matriz de Decisões Técnicas

6.1 Por Que FastAPI para o Gateway (e Não n8n)

Critério	FastAPI Gateway	n8n
RAM	30-80 MB	300-600 MB
Latência	< 1ms	5-50ms
Customização	Código livre	Limitado à UI
Versionamento	Git	JSON export
Testing	pytest nativo	Manual
Custo VPS	Mínimo	Alto

Decisão: FastAPI como padrão. n8n como módulo opcional para clientes não-técnicos.

6.2 Por Que Next.js para o Frontend (e Não React puro)

Critério	Next.js	React + Vite
SEO	SSR/SSG nativo	CSR only
BFF	API Routes integrado	Precisa proxy
Deploy	Vercel ou auto-hospedado	Precisa Nginx
shadcn/ui	Suporte nativo	Manual
Performance	Edge + ISR	SPA only

Decisão: Next.js como padrão para benefícios de BFF e SEO.

6.3 Banco Centralizado vs. Multi-Instance

A v1 ja definia o **Cenário A** (PostgreSQL centralizado). A v2 confirma:

Componente	Descricao
PostgreSQL (postgres:16-alpine)	1 instancia centralizada
twenty_db	Banco logico - CRM
chatwoot_db	Banco logico - Atendimento

Componente	Descricao
calcom_db	Banco logico - Agendamento
+ outros	Conforme nicho selecionado

Servicos conectados: Twenty CRM, Chatwoot, Cal.com, e demais ferramentas do nicho.

Vantagens confirmadas:

- Economia de 1-2 GB RAM vs múltiplas instâncias
- Backup consolidado via `pg_dumpall`
- Isolamento lógico por schema/banco

7. Gaps Não Abordados na v1 (ou Abordados Superficialmente)

7.1 Busca Dinâmica no GitHub (Gap Crítico)

v1 dizia: "Subagente 1 consulta a base de dados vetorizada e o GitHub via MCP"

Realidade: O `catalogo_nichos.json` é estático com 5 nichos e 11 ferramentas

Solução v2: O AIDD-Factory inclui um **Discovery Engine** que:

- Recebe o texto do negócio
- Busca no GitHub API (repositórios > 100 stars, atividade < 90 dias)
- Filtra por: licença permissiva, imagem Docker oficial, documentação
- Valida contra o catálogo existente (override ou adição)
- Gera candidatos para curadoria humana

Isso **não pertence ao aidd-ops** (que é determinístico) nem ao aidd-generator (que gera código Python). É responsabilidade do factory.

7.2 Autenticação SSO/IAM (Superficial na v1)

v1 dizia: "Authentik / Keycloak com OIDC/OAuth2"

Realidade: Template Authentik existe mas não há integração automática

Solução v2:

- Template `authentik/docker-compose.yml` já existe e funciona
- A Fase 6 (`init.sh`) deve criar o realm OIDC automaticamente
- O gateway deve validar tokens JWT via middleware
- O frontend deve redirecionar para Authentik no login

7.3 Backup e Disaster Recovery (Não Implementado)

v1 dizia: "Dumps diários automáticos para Cloudflare R2"

Realidade: Nenhum código de backup existe

Solução v2: Adicionar ao pipeline do factory:

Cron job de `pg_dumpall` → compactação → upload R2

Script de restore testado

Validação de integridade do dump

7.4 Observabilidade e Alertas (Parcial)

v1 dizia: "Uptime Kuma + Dozzle com notificação via Webhook"

Realidade: Uptime Kuma export/check funciona, mas sem notificação

Solução v2:

Uptime Kuma já suporta notificações nativas (Telegram, WhatsApp, email)

O factory deve gerar a configuração de notificação

Dozzle para logs em tempo real (leve, ~10 MB RAM)

7.5 Escalabilidade e Multi-Tenant (Não Abordado na v1)

Gap novo: A v1 não aborda como múltiplos clientes operam na mesma VPS

Solução v2:

Cada cliente = namespace Docker separado

PostgreSQL: schemas isolados por tenant

Gateway: routing por subdomínio (`tenant.app.exemplo.com`)

Frontend: dynamic tenant injection via `tenant.config.json`

8. Roadmap de Implementação

Fase 1: Fundação (2 semanas)

- [] Criar `tools/aidd-factory/` com estrutura base
- [] Migrar `result.py` de `componentes/compartilhado/`
- [] Implementar `pipeline_factory.py` (CLI Click)
- [] Implementar Fase 1 (analisador determinístico)
- [] JSON Schema de input/output
- [] Gate `G_FACTORY_INPUT.py`

Fase 2: Gateway & Docker (3 semanas)

- [] Templates Jinja2 para FastAPI gateway
- [] Engine de geração `gateway_generator.py`
- [] Fase 4 (compose unificado determinístico)
- [] Fase 5 (`init-multiple-databases.sh`)
- [] Fase 6 (`.env` generation)
- [] Gate `G_FACTORY_COMPOSE.py`

Fase 3: Frontend Whitelabel (3 semanas)

- [] Templates Jinja2 para Next.js

- [] Engine de geração `frontend_generator.py`
- [] `tenant.config.json` injection
- [] BFF routes auto-generated
- [] `shadcn/ui` component library

Fase 4: Integrações (2 semanas)

- [] Mapeamento de integrações por nicho (`data/integracoes.json`)
- [] Fase 7 (webhook generation)
- [] Fase 8 (Swagger/OpenAPI)
- [] Fase 9 (cross-service validation)

Fase 5: Orquestração Cross-Tool (2 semanas)

- [] Integração `aidd-ops` → `aidd-factory` → `aidd-ops`
- [] CLI unificada: `ecossistema.py factory "" --deploy`
- [] Testes E2E completos
- [] Documentação

Total estimado: 12 semanas (1 pessoa) ou 6 semanas (2 pessoas)

9. Contratos entre Ferramentas

9.1 `aidd-ops` → `aidd-factory`

Input: `PLANO-INFRAESTRUTURA.json`

Output: `FACTORY_ANALYSIS.json`

O `PLANO-INFRAESTRUTURA.json` é o contrato canônico. O `factory` não precisa conhecer o catálogo de nichos — ele recebe o plano já resolvido.

9.2 `aidd-factory` → `aidd-ops`

Input: `FACTORY_OUTPUT.json` (lista de artefatos gerados)

Output: Deploy Orchestration

O `factory` produz um diretório pronto com:

```
docker-compose.yml unificado
.env + .env.enc
init-multiple-databases.sh
src/gateway/ (FastAPI)
frontend/ (Next.js)
openapi.json
README.md
```

O `aidd-ops` faz deploy desse diretório via Coolify ou Docker Compose direto.

9.3 `aidd-generator` ↔ `aidd-factory`

O generator NÃO se conecta ao factory.

São ferramentas para propósitos distintos:

generator: código Python avulso (CLI, scripts, ferramentas)

factory: código de aplicação multi-serviço (gateway, frontend, integrações)

10. Análise de Risco

Risco	Probabilidade	Impacto	Mitigação
LLM gera código inseguro no gateway	Média	Alto	Gate AST + bandit scan + testes E2E
Compose gerado tem conflito de portas	Baixa	Alto	compose_preflight.py + G_FACTORY_COMPOSE.py
Frontend whitelabel não compila	Média	Médio	Template testado + CI gate
Webhook contracts inconsistentes	Média	Médio	JSON Schema validation + cross-service test
Custo de tokens LLM muito alto	Baixa	Médio	Budget por fase + fallback determinístico
Multi-tenant isolation é inseguro	Baixa	Crítico	Pen test + network isolation + SQL injection gates

11. Conclusão

A v1 deste documento era um plano ambicioso e correto em sua visão. Sua limitação foi subestimar a complexidade de **gerar código de integração e frontend** como parte da infraestrutura.

A v2 propõe:

1. **Manter o aidd-ops intocado** — ele é excelente no que faz
2. **Criar o aidd-factory** — como ponte entre planejamento e deploy
3. **Integrar as 3 ferramentas** — num pipeline de 15 fases com contratos claros

O resultado é um ecossistema onde:

aidd-ops resolve "*como coloco em produção?*"

aidd-factory resolve "*o que como integro?*"

aidd-generator resolve "*gero código Python genérico?*"

Juntos, transformam **dores de negócio em plataformas completas, soberanas e operacionais** — exatamente como a v1 prometia, mas com a arquitetura certa para entregar.